الطلب المزوّر من الموقع المتنمّر

Lecture 7 — Cross-Site Request Forgery (CSRF)

> **Cross-Site Request Forgery (CSRF)** : a malicious page makes the victim's browser send a **state-changing request** to a site where the victim is logged in. The browser attaches the session cookie automatically, so the site thinks the victim meant it.

CSRF	XSS
Attacker's code runs on another site	Attacker's code runs **on the target site**
Abuses the browser's automatic cookies	Injects script into the page
Can't read responses	Can read the page, tokens, cookies (if not HttpOnly)
Defense: tokens, SameSite, Origin check	Defense: output encoding, CSP

> **Ambient authority** : performing an action just by naming the resource, without presenting a permission scoped to that action. Cookies work this way: attached by destination domain, **never checking which page initiated** the request.

> **Confused deputy problem** : a program holding more authority than the requester is tricked into misusing it. In CSRF: the **browser** is the deputy, the **session cookie** is the authority, the **malicious page** tricks it.

Tokens work because they replace implicit (ambient) authority with an **explicit, per-action permission** only the legitimate origin can obtain.

State-changing GET requests

Q/ Why is a state-changing GET dangerous ?

1. GET is supposed to be **safe** (no state change).
2. A forged GET needs only an **image tag or a link** — the browser sends it as soon as the tag is parsed; no script, no click.

3. A forged POST needs at least an auto-submitting form; exposing a transfer or password change via GET removes even that step.

Primary defenses

No single defense is complete. Layer a **token** with **SameSite**: SameSite is the safety net; the token holds when the net doesn't apply.

Defense	Mechanism	Limitation
Synchronizer Token Pattern	Server generates a unique, unpredictable token per session, embeds it in every state-changing form, rejects mismatches	Server-side state; must be in every form/API call
Double submit cookie	Random value set as a cookie **and** repeated as a request parameter; another origin can't read the cookie to copy it	Naive version weak unless **signed and session-bound**
Custom request header	Script adds a custom header; only same-origin script can set it	Not usable for plain forms or links
SameSite cookie	Browser withholds the cookie on most cross-site requests	**Lax** (default) still sends it on top-level navigation like a clicked link
Origin / Referer check	Compare the declared source with your own origin	Referer can be stripped by Referrer-Policy or privacy settings

> **Naive double submit** : assumes an attacker can't set cookies on your domain — broken by a vulnerable subdomain, **subdomain takeover**, or plain-HTTP access.

> **Signed double submit cookie** : value generated with a server secret (**HMAC**) and **bound to the session ID**; a planted cookie can't pass the signature check. (Current OWASP guidance.)

Origin header	Referer header
Scheme + host + port only (no path)	**Full URL** of the requesting page
Sent on all cross-origin requests and same-origin POST/PUT/DELETE	Can be suppressed by Referrer-Policy or browser settings

Origin header	Referer header
More dependable — check first	Fallback when Origin is absent

SameSite in practice

Q/ When does SameSite=Lax (the default since 2020) still allow a forged request ?

1. **Top-level GET navigation** — a link the victim clicks — so a state-changing GET is still forgeable.
2. Chromium's **temporary exception**: a cookie **under 2 minutes old** is sent on a top-level cross-site **POST** (kept for SSO/identity flows; expected to be removed).

SameSite is a valuable **default, not a substitute** for an explicit token.

XSS defeats CSRF tokens

Q/ Why can an XSS vulnerability defeat token-based CSRF protection ?

- Injected script runs with the page's full privileges, so it can **read the token** from the page (or a readable cookie) and copy it into the forged request. One XSS removes the protection against a completely different attack.

> **OWASP placement** : CSRF was **#5 (2007)** and **#8 (2013)**, then **dropped from 2017** (declining incidence; frameworks added built-in protection). In 2021 it is **CWE-352**, included under **A01:2021 Broken Access Control** — processing a state-changing request without verifying the user's genuine intent is a failure to control access to that action.

Summary of Web Applications Security L7: Cross-Site Request Forgery (CSRF). Built only from the lecture file; check the original slides for anything you're unsure about.